

Raytracer Project Report

Eugenio Animalì

1 Introduction

This report describes the steps I took to build a raytracer. I started with basic sphere intersection and direct lighting, then added shadows, mirror reflection, indirect lighting, antialiasing, and was able to implement intersection with single triangle objects.

2 Light source and Shadows

The first working version has a scene with one sphere that distinguishes light areas, shadow areas and areas that never intersect an object and can be seen in Figure 1.

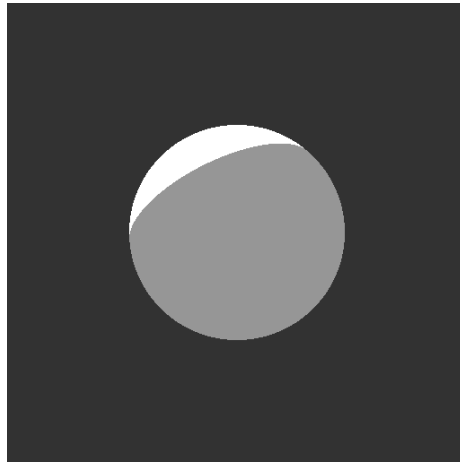


Figure 1: External light

This required implementation of a Sphere object's intersection with a ray, which was done making sure to return a result as soon as it was found to avoid unnecessary computations; as well as the Scene's getColor function. The getColor function identifies the intersection with each candidate object, and in the case of more than zero intersections, identifies whether the first (by distance

from the camera) is in the line of sight of the light source by sending a new ray from a small epsilon above the surface towards the light and checking for no intersections between them. At this stage a white color was returned if the light was visible, grey if there was a shadow, and black if there was no intersection.

However, due to some confusion regarding how far the camera was to the pixel grid, I was getting warping of the sphere when I moved it from the center of the image. This was fixed by adjusting the position of the camera (the origin of each ray) to respect a 60 degree fov, rather than a fixed distance of 55.

3 Mirror Surface and Scene

Mirrors were a simple addition, as the ray hitting the surface of the mirror is redirected across from the normal of the sphere at the intersection point (accounting for floating point inaccuracy by starting the ray an epsilon above the surface of the sphere), and the color is returned exactly to the camera. With returning albedo instead of simply white light, and solving problems associated to iterating through objects in the scene, I was able to generate Figure 2, with one opaque white ball and a smaller mirror just behind it to the right, as well as six walls to create a room.

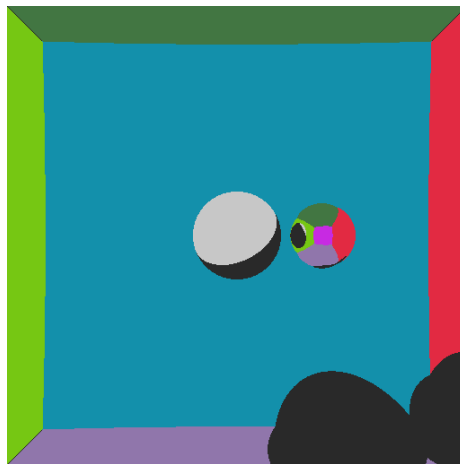


Figure 2: Mirror and Multiple objects

Up to this point, all the colors returned are indiscriminate of light intensity or distance from the light source, and appear as fill colors in the regions they create. However, to make a more realistic scene, light bouncing off a surface must be influenced by the relative angle of the normal of the surface and the incoming light ray.

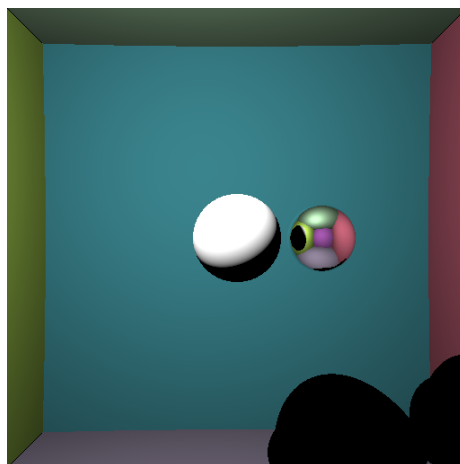


Figure 3: Diffuse Lighting

4 Diffuse Lighting

In Figure 3 we have the same scene, where the light returned by `getColor` respects the Lambertian model for reflection off matte surfaces like plaster and plastic. This does not depend on the reflected angle, as light is set to irradiate uniformly off matte surfaces, but as the normal of the surface moves away from the line of the incoming light, the light gets spread over a larger area, and thus results in a lower reflected intensity.

5 Indirect Lighting and Antialiasing

I implemented indirect lighting using Monte Carlo sampling. For diffuse surfaces, instead of just returning the direct lighting, I also shoot a random ray in the hemisphere around the normal, distributed by a random distribution based on the Box-Muller transform as BRDF and accumulate the incoming light. This makes the lighting look softer and more natural, as well as adding natural lighting to shadows. Notice that the corners of the room in Figure 3 are very dark as the normal of the surfaces is almost perpendicular to the light source. This is a common issue that adding indirect lighting fixes, as, like in the real world, most of the light these places get is due to reflections from other surfaces. We can choose how many times indirect light can bounce off surfaces from the light source before being truncated, as the component from a subsequent reflection is always diminishing, and by the second or third bounce, the change from the light added is often indistinguishable.

Although the indirect light that a surface point P receives in the direction of a ray is in reality an integral of all different directions off P such that the scalar product with the normal N is positive, the antialiasing that we also im-

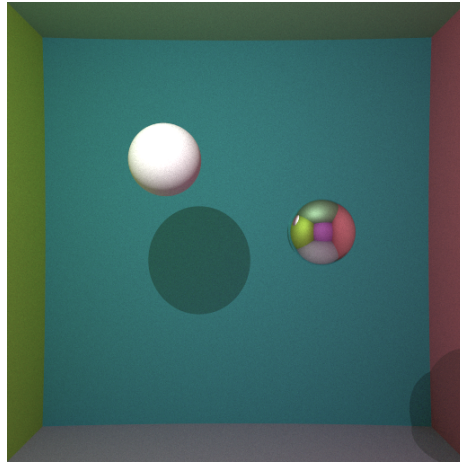


Figure 4: Indirect Lighting and Antialiasing

plement in this section means that for each pixel, we are summing the indirect light of several rays, such as Monte Carlo integration, allowing for an efficient computation with low noise.

With antialiasing, I shoot multiple rays per pixel (500 in Figure 4) with small random offsets to the ray direction (using a box-muller distribution to keep samples within a small area around the center of the pixel). The final pixel color is the average of these samples.

6 Triangle Intersection

The last feature I worked on was triangle intersection. I did not complete full triangle meshes with BVH, but I implemented intersection for a single triangle. The triangle is defined by three vertices, and I used the ray-triangle intersection algorithm based on barycentric coordinates. Figure 5 shows the scene with one triangle added among the spheres, and an adjusted gamma value.

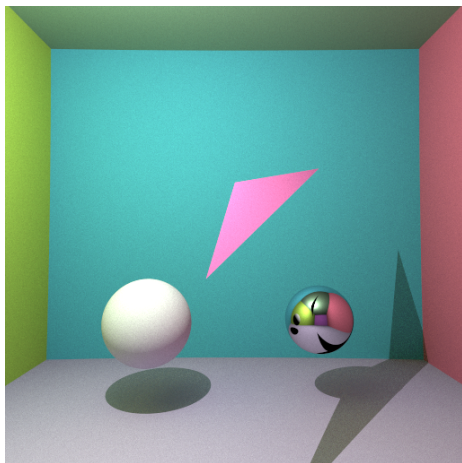


Figure 5: Triangle interacting with the Scene (512p x 512p)

Figure 5 took 133.077 seconds to produce with 1000 rays per pixel while Figure 6 took 1.205 seconds to produce with 10 rays per pixel.

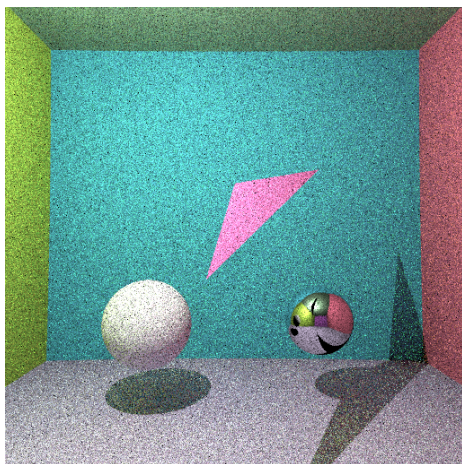


Figure 6: Noisy Image (512p x 512p)